

Analysis of a Fast Inverse Square Root Algorithm

MET CS566 - Analysis of Algorithms

Fall 2023

Presentation by Joseph Rissman

U09545917

Table of Contents

Introduction

Motivation

The Code

Evil Bit Hack

W.T.F.

Newton's Method

Benefits

Conclusion



Introduction

- An ingenious technique for quickly calculating inverse square roots became famous after it was discovered in the source code of a video game from 1999 called Quake III Arena, published by id Software.
- The method was previously published in 1997 by Jim Blinn and has been used in many 3D computer graphics software engines in the late 90s and early 2000s before the invention of specialized hardware.
- I will compare the fast inverse square root algorithm to the other methods available at the time in terms of time complexity and accuracy.

Motivation

- In 3D computer graphics, it's crucial to find the normal vector for surfaces to compute angles of incidence and reflection for lighting and shading. Millions of calculations are performed each second to simulate lighting.
- The normal vector $v = (v_1, v_2, v_3)$ must be scaled to length 1. The following equation is used to find the normalized unit vector $\hat{v}$:

$$\hat{v} = v \frac{1}{\sqrt{v_1^2 + v_2^2 + v_3^2}}$$

- In the 90s, computer hardware was able to perform floating-point multiplication and addition very quickly, but floating-point division was computationally very expensive. Therefore, calculating $v_1^2 + v_2^2 + v_3^2$ is fast, but a clever algorithm was required to bypass the division step.

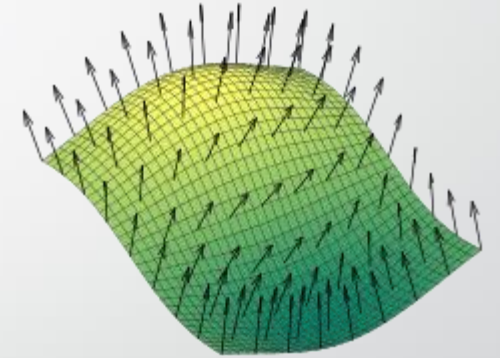


Diagram depicting normal vectors of a surface.

The Code

```
float q_rsqrt(float number)
{
    long i;
    float x2, y;
    const float threehalfs = 1.5F;

    x2 = number * 0.5F;
    y = number;
    i = * ( long * ) &y;           // evil floating-point bit level hacking
    i = 0x5f3759df - ( i >> 1 );  // what the f@$#!?
    y = * ( float * ) &i;
    y = y * ( threehalfs - ( x2 * y * y ) ); // 1st iteration
    // y = y * ( threehalfs - ( x2 * y * y ) ); // 2nd iteration, this can be removed

    return y;
}
```

The code to the left is copied exactly from the *Quake III Arena* source code, written in C, including the original comments (with profanities removed).

This algorithm works for floating-point numbers, which all 3D graphics software use to represent vectors, and returns a floating-point number that is equal to

$$\frac{1}{\sqrt{x}} = x^{-\frac{1}{2}}$$

The algorithm computes the inverse square root by the following steps:

1. Bit manipulation to approximate $\log_2(x)$.
2. Use the approximation to find $\log_2\left(\frac{1}{\sqrt{x}}\right) = -\frac{1}{2}\log_2(x)$.
3. Reverse step 1 to approximate base-2 exponentiation.
4. Use Newton's method to improve the approximation.

Evil Bit Hack

For mathematical reasons that are beyond the scope of this presentation, a floating-point number following IEEE 754 format has a special property. The integer value of the bits representing the floating-point number x is approximately equal to $\log_2(x)$. That is why this line of code is necessary to convert the floating-point number into an integer without changing any of the bits.

```
i = * ( long * ) &y;          // evil floating-point bit level hacking
```

This step immediately computes an approximation for the base-2 logarithm of the number and skips the computationally expensive step of finding an approximation in a lookup table. Now the approximation for $\log_2(x)$ can be used to compute the inverse square root of the number in an efficient manner.

Although this algorithm was originally designed for 32bit floating-point numbers following the IEEE 754 standard, it was later shown to work with other floats.

W.T.F.

Now that the base-2 logarithm of the number has been computed, it's used to calculate the inverse square root in this seemingly magical line of code.

```
i = 0x5f3759df - ( i >> 1 );           // what the f@$#?
```

What's actually happening here is some more bit manipulation to compute $-\frac{1}{2}\log_2(x)$, which is very close to the desired value.

The integer which is the approximation of $\log_2(x)$ is bit shifted right once which is equivalent to dividing by 2 but eliminates the costly division step. The integer is subtracted from the 'magic number' to make the value negative. The bit shift and negation effectively multiplies the integer (i.e., approx. $\log_2(x)$) by $-\frac{1}{2}$.

We now have $-\frac{1}{2}\log_2(x) = \log_2\left(\frac{1}{\sqrt{x}}\right)$.

W.T.F

What about the mysterious hexadecimal constant that seemed to incite profanity in the code's comments?

The constant `0x5f3759df` is the floating-point number representation of $\sqrt{2^{127}}$ which is a correction term that needs to be added back due to all the bit manipulation that occurred to approximate $-\frac{1}{2}\log_2(x)$. Once again, all of the bit manipulation details are outside the scope of this presentation.

Once the approximation $\log_2\left(\frac{1}{\sqrt{x}}\right)$ had been calculated, the result was converted back into a floating-point number without altering the bits to approximate a base-2 exponentiation. This removes the logarithm and provides a floating-point number approximately equal to $\frac{1}{\sqrt{x}}$.

Newton's Method

Although the bit manipulation methods allow for a fast approximation of $\frac{1}{\sqrt{x}}$, it isn't accurate enough for 3D software to render lighting on surfaces. To achieve a higher level of accuracy, Newton's method is applied.

Newton's method is used to find better approximations of the zeroes of any function for which the derivative is known. The method can be applied repeatedly to find better and better approximations, but for the inverse square root, only one iteration is necessary to get accuracy to within 0.1% of the true value.

Newton's Method

Newton's method uses the first derivative to find the tangent at the current value x_0 which is then used to calculate the next value x_1 which will be a better approximation of the zero of the function. It follows the general formula

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

For the inverse square root, the function to find the zero of is

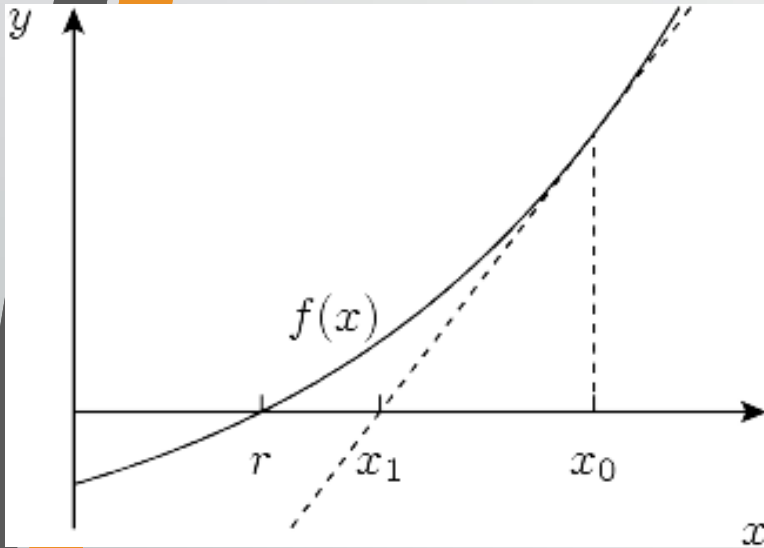
$$f(y) = \frac{1}{y^2} - x = 0$$

where y is the inverse square root and x is the input number. Finding the derivative and substituting both into the equation for Newton's method gives the following formula

$$y_{n+1} = y_n \left(\frac{3}{2} - \frac{x}{2} y_n^2 \right).$$

This is written in the algorithm as this line of code which again avoids the costly divisions by clever use of variables

```
y = y * ( threehalfs - ( x2 * y * y ) ); // 1st iteration
```



Graphical representation of Newton's method.

Benefits

Unfortunately, it is no longer possible to accurately demonstrate the performance improvement of the fast inverse square root algorithm compared to the 'built-in' methods because since the introduction of machines with the x86 instruction set, processors have a dedicated inverse square root method which is more accurate and faster than the fast method described here.

An article published by Elan Ruskin in 2009 compared the performance of several square root operations, including the fast inverse square root algorithm. The x87 FPU instruction set, which is what was available when the fast inverse square root algorithm was devised, takes approximately 5-6 times as long to compute compared to the fast inverse square root method. In the same article, the inverse square root calculated with an x86 instruction set was able to execute in $1/4^{\text{th}}$ the time required for the fast inverse square root algorithm and is roughly 10x more accurate.

Conclusion

- The fast inverse square root algorithm can improve performance of 3D graphics engines at a modest cost of accuracy by taking advantage of floating-point number representation and mathematical/bit manipulation tricks to eliminate costly operations such as table lookups and floating-point number divisions.
- Although the algorithm quickly became obsolete due to improved built-in methods in x86 instruction sets, it provides an excellent example of how a thorough understanding of mathematics and bit manipulations can be used to optimize code. Without this algorithm, there would have been no hope for these old video games to run on the average computer of the time.

References

- Blinn, Jim (July 1997). "Floating Point Tricks". IEEE Computer Graphics & Applications. **17** (4): 80. [doi:10.1109/38.595279](https://doi.org/10.1109/38.595279).
- Hellman, Noah (2023, April 20). *Revisiting The Fast Inverse Square Root - Is It Still Useful?*. hllmn.net. https://hllmn.net/blog/2023-04-20_rsqrt/
- Moroz, Leonid V.; Walczyk, Cezary J.; Hrynychyshyn, Andriy; Holimath, Vijay; Cieslinski, Jan L. (January 2018). "Fast calculation of inverse square root with the use of magic constant analytical approach". *Applied Mathematics and Computation*. Elsevier Science Inc. **316** (C): 245-255. [doi:10.1016/j.amc.2017.08.025](https://doi.org/10.1016/j.amc.2017.08.025).
- Ruskin, Elan (2009 October 16). *"Timing square root"*. Some Assembly Required. Archived from [the original](#) on 2021-02-08.